PROJECT PLAN | 14 OCTOBER 2026

GTA V on iPhone
Native development roadmap

The shortest verified path from the current native prototype to playable Prologue, then Sandbox and better image quality.

First delivery target

An experimental native IPA that runs the existing engine and local game assets on the iPhone 16 Pro Max / A18 Pro, renders the Prologue, accepts player input and progresses through the mission. Networking and high settings are deferred.

Where we stand

Stages 1 and 2 are complete. Stage 3 is active. Five stages remain, including stage 3. Native 0.18 is built and audited. Its physical iPhone scene test is pending; native gameplay, responsive controls and audio output are not yet verified.

Stage	Status	Completion gate
1. Baseline and dependency map	Completed	Reproducible iPhone runtime, assets, saves and worker checks.
2. Renderer feasibility	Completed	Original game shader draws produce checked pixels on iPhone.
3. Actual Prologue scene	In progress	Successive correct 3D scene frames, camera and animation.
4. Controls, audio, lifecycle	Planned	Responsive input, mission sound, pause/resume and safe shutdown.
5. First playable build	Planned	Repeatable 10-minute Prologue route and mission transition.
6. Sandbox and stability	Planned	Walking/driving routes and sustained 20-30 minute sessions.
7. Quality improvements	Planned	Visible improvements that retain measured stability and performance.

Critical path

Verified runtime and renderer > actual Prologue scene > input and audio > sustained playable Prologue > Sandbox > quality tuning.

The first playable test is delivered at stage 5. It does not wait for open-world population or higher resolution. Completion dates will be reassessed after the first verified scene; compiling shaders alone cannot establish the remaining runtime work.

STAGES 1 AND 2 COMPLETED

Completed foundations

Stage 1 | Baseline and playable path mapped

Purpose: establish that the actual engine can run natively and access the selected game files, with the startup dependencies understood before broad integration.

1. **Validate the runtime.** Check real engine constructors, main-thread and worker TLS, shared memory, clocks, environment, file ABI and persistent save I/O.
2. **Validate local assets and workers.** Index the selected web folder, check queued head/tail reads, EOF and invalid requests, then exercise concurrent reads, mailbox/proxy replies and joined worker cleanup.
3. **Map the real startup protocol.** Inspect independent engine/JavaScript references for host imports, GPU command layouts, synchronization, shader inputs and resource lifetimes. Classify implemented, untested and missing behaviors.
4. **Establish a phone baseline.** Collect two physical 0.4 reports. Both passed the asset/worker checkpoint and reached the first archive request, stopping deliberately before its bytes.

Gate passed: reproducible runtime/asset/worker checks and a concrete startup dependency map. These early memory samples are not a gameplay memory budget.

Stage 2 | Native renderer feasibility

1. **Select one renderer route.** Compare native WebGPU on Metal with a direct Metal adapter using the actual protocol, shader compatibility, iOS ARM64 support, licensing, package size and memory cost. The selected route is wgpu-native on Metal.
2. **Prove original shader output.** Replay original postfx vertex/fragment shaders with the packed constant layouts, texture/sampler binding and changed uniforms. Compare all 4,096 pixels per draw within the recorded tolerance.
3. **Check device and cleanup.** Require zero validation/runtime errors and repeat the physical iPhone renderer test. Two 0.5 iPhone runs passed, alongside the CPU regression checkpoint.

Gate passed: a representative original shader/draw works on the A18 Pro GPU. This established renderer feasibility; it did not establish a rendered game scene.

What later builds have established

Real engine startup has advanced from the blue boot screen through legal notices to the North Yankton intro path. Native 0.17 executed an actual depth-downsample compute dispatch before the clip-distance failure. Native 0.18 fixes that failure and passes the full translated shader audit. Physical scene continuation is still required.

Keep the successful diagnostic checkpoints and prior IPAs available while the integrated startup path changes. A regression can then be isolated without rebuilding the foundations.

IN PROGRESS | CURRENT NEXT GATE

Stage 3 | Render the actual Prologue

Purpose: turn the verified native runtime and renderer into successive correct game frames containing the world, mission actors, textures, moving camera and animation.

1. **Continue from native 0.18 on the phone.** Use the existing selected web folder and conservative 640 x 360 profile. Run Trace Prologue startup with the game view visible. Check what appears after North Yankton, then preserve the full final report.
2. **Finish required graphics behavior.** Follow the real shared command stream through formats, shader combinations, vertex/instance inputs, depth proxies, UAVs/compute, copies, readbacks and resource destruction. Implement correct ordering and GPU completion before reuse. Unsupported operations must report a specific failure.
3. **Finish required host and worker behavior.** Implement startup services and asynchronous completions as the engine reaches them. Preserve mailbox delivery during waits and shutdown, cancellation behavior and joined cleanup. Alternate host and renderer fixes as dependencies require.
4. **Diagnose loading and metadata.** Correlate scene-sphere requests with pending assets, delivered bytes, script phase and camera start. Investigate scenario class/type metadata, defaulttasks registration, missing audio banks and interiors against the selected build. Do not fabricate classes or report missing assets as loaded.
5. **Prove scene output and animation.** Use the visible phone scene and bounded pixel diagnostics together. Confirm consecutive frames contain actual actors/world with correct camera, textures and animation, rather than treating draw/presentation counters as proof.
6. **Measure the real startup cost.** Record peak process footprint, engine heap, GPU allocation trends, frame times, compile stalls and I/O. Preserve bounded reads: one 64 KiB reader buffer and at most eight open files; avoid whole-archive caching and duplicated assets.

Completion gate

The actual engine loads Prologue and presents successive correct game-scene frames on the iPhone, with startup peaks and frame timing recorded. The visual gate does not yet claim playable input or native audio.

Latest fixes and limits

0.18 passed: 4,808 translated shader modules and native pipelines, including 500 clip shaders; 24 clip pixel cases; retained rendering/runtime regressions and the unsigned ARM64 IPA audit. It fixes clip feature negotiation, Metal clip-array emission, stale backend dependency caches and integer/null multisample bindings.

Still unverified: real shader pairs and dynamic bindings across gameplay, 111 untranslated hull/domain/geometry records, guarded graphics operations and the actual iPhone scene. Metadata, movie, network/logging and audio warnings remain distinct from the resolved GPU failures.

Current status: (stage 3 in progress) (5 stages left)

PLANNED | BEGINS AFTER THE SCENE GATE

Stage 4 | Controls, audio and lifecycle

Purpose: make the rendered mission respond to the player, produce correctly timed sound and behave safely when the app is paused, interrupted or closed.

1. **Connect touch controls.** Feed movement, camera look and essential mission actions into the real engine input state. Include pause/menu controls and simultaneous actions. Confirm movement and camera response in the scene, not just a button indicator.
2. **Connect Bluetooth controllers.** Use Apple GameController for supported paired controllers. Prefer event-based updates over idle polling, update prompts on actual input, and clear held actions on disconnect or backgrounding. Start with one available controller; broaden coverage after the playable milestone.
3. **Consume the engine audio queue.** Use bounded native output buffers. Keep audio callbacks free of disk I/O, blocking locks and routine allocations. Verify sample format, timing, queue flow and underruns before judging dialogue/cutscene synchronization.
4. **Check mission sound and progression.** Exercise dialogue, effects and cutscene audio in Prologue. Investigate missing banks and the reported zero-duration synchronized audio path where they affect playback or progression. A passing GPU test cannot validate sound.
5. **Handle app interruptions.** Pause/resume coherently, release stale held inputs and handle controller reconnects and audio interruptions. Stop startup and join workers without deadlocks or resource reuse while GPU work remains in flight.
6. **Retain folder access and saves.** Restore selected-folder access where supported; report expired permissions clearly. Keep writable saves inside the app and game data read-only. Preserve the working persistent-save and traversal-rejection checks.

Completion gate

Player movement, look and actions respond in the actual game; mission audio progresses correctly; pause/resume and shutdown work. Controller behavior is verified separately on the physical phone.

Resource budget

Input handling should not create an independent busy loop. Reuse fixed state/buffers, keep audio queues bounded and avoid per-frame filesystem work or controller discovery. Measure overhead with the game running before adding optional control features.

What this stage delivers

An integrated input/audio/lifecycle build ready for the repeatable gameplay route in stage 5. Record which touch actions and controller model were exercised, audio limitations and interruption behavior. A controller button probe alone does not close this stage.

PLANNED | FIRST PLAYABLE DELIVERY

Stage 5 | First playable iPhone build

Purpose: deliver a conservative experimental Prologue build after testing real mission play, rather than waiting for Sandbox or higher settings.

1. **Define a repeatable route.** Start Prologue, exercise movement/look/actions, play through the vault/door/cutscene transition that crashed the browser version, and continue into the following gameplay section. Record the route and any required actions.
2. **Run at least 10 minutes.** Keep the scene and controls active through a sustained session. Include pause/resume and repeat cold starts. Test save/load when the mission allows it; do not infer it from the earlier probe-save check.
3. **Measure peaks and trends.** Capture frame times, startup and gameplay memory peaks, engine heap and GPU resource changes. Compare app headroom and OS pressure across runs. Cleanup samples alone cannot establish the in-game budget.
4. **Separate bottlenecks.** Distinguish CPU work, GPU work, I/O, shader compilation, synchronization and memory pressure. Fix reproducible resets, missing visuals, deadlocks and mission-progression blockers before changing the quality preset.
5. **Optimize measured hot paths.** Reduce redundant state changes, copies and allocations only where profiles justify them. Compare compiler optimizations while preserving ABI, atomic ordering and floating-point semantics; keep relevant correctness checks.
6. **Package the playable test.** Build and audit the unsigned ARM64 IPA through GitHub Actions. Preserve the previous working IPA, document the tested route, device, performance and limitations, and install through the established local signing/sideloading route.

Completion gate

A repeatable 10-minute Prologue route, including the vault/door transition, works with actual rendered scenes and responsive controls. Deliver an EXPERIMENTAL PLAYABLE PROLOGUE IPA with measured performance and explicit remaining limits.

Performance target

Target stable 30 FPS, then report what the phone actually achieves: typical frame time, slow frames, stalls, memory peaks and any thermal slowdown. Passing the route matters more than a high average FPS over a short or idle scene.

Release evidence

Retain the exact source revision, CI run, dependency provenance, IPA digest and full test report. Distinguish a build/package pass from phone gameplay verification. Do not describe the build as playable until the route and input checks pass.

PLANNED | AFTER PLAYABLE PROLOGUE

Stage 6 | Sandbox and sustained stability

Purpose: extend the working native game to open-world travel, then introduce pedestrians and traffic without losing the tested Prologue baseline.

1. **Enable conservative Sandbox startup.** Keep the established rendering/profile baseline while changing the startup path. Confirm world loading, camera, collision and controls before increasing population or resolution.
2. **Test fixed travel routes.** Walk and drive through defined areas, recording scene changes, streamed resources and repeat visits. Reuse the routes to distinguish travel-related failures from random differences between sessions.
3. **Add pedestrians gradually.** Introduce low density in a separate change. Check the scenario metadata/class diagnostics against actual spawning and behavior, plus animation, collision and streaming demand. Measure CPU, heap and GPU changes.
4. **Add random traffic gradually.** Start with low density after the walking/pedestrian baseline. Verify vehicle models, movement, collision and streaming while driving. Keep population increments separate from image-quality changes.
5. **Run 20-30 minute sessions.** Measure heat-related slowdown, allocation trends, repeated scene loading, cold launches, pause/resume and controller reconnects. Repeat routes so retained memory growth can be distinguished from temporary loading peaks.
6. **Fix long-session problems.** Address retained resources, temporary allocation spikes, queue pressure and shader stalls. Bound caches and retire or evict only resources no longer used by the engine or pending GPU work. Preserve the known stable preset.

Completion gate

Repeatable Sandbox walking/driving works through sustained sessions without resets, deadlocks or continuing retained-memory growth on the tested routes. Document the tested population levels and remaining limits.

What will stay conservative

The first Sandbox check uses the proven quality baseline. Pedestrian density, traffic density, resolution and effects each have different costs; changing them together hides the source of failures. Keep a working rollback artifact after each successful increment.

Expected delivery

An experimental Sandbox IPA with tested routes and density limits. This stage verifies useful open-world play; it does not promise every mission, area or maximum population on the phone.

PLANNED | AFTER SUSTAINED STABILITY

Stage 7 | Better quality within measured limits

Purpose: use the phone's demonstrated headroom to improve image quality while retaining a tested conservative preset.

1. **Establish comparable captures.** Use identical Prologue and Sandbox routes for each before/after comparison. Record typical and slow frame times, CPU/GPU timing where available, memory peaks, thermal state and visible differences.
2. **Raise resolution first, independently.** Increase resolution in controlled steps without changing pedestrians or traffic. Check actual GPU frame cost and render-target memory. Roll back if frame pacing, heat or sustained stability regresses.
3. **Tune population separately.** Use the stable stage 6 density baseline and measured CPU/streaming cost. Increase only where the route and long-session checks preserve responsiveness and memory behavior.
4. **Tune assets and effects.** Evaluate textures, draw distance and individual effects one at a time. Preserve mission geometry and required actors/resources. Prefer settings with a visible benefit at an acceptable measured cost.
5. **Optimize the identified bottleneck.** Reduce redundant pipeline/state work, uploads, copies and CPU/GPU waits where profiles identify them. Use bounded cache policies with correct GPU lifetime handling rather than unbounded retention.
6. **Consider adaptive resolution conditionally.** Add it only if sustained measurements show GPU load is the limiting factor and the quality tradeoff is useful. It will not repair a CPU or engine-heap bottleneck.
7. **Publish tested presets.** Keep a conservative fallback and document the better-quality preset's device, routes, duration, frame timing, peak memory and thermal behavior. Preserve rollback after regressions.

Completion gate

Visible improvements survive repeatable and sustained route testing, with measured performance and a stable fallback. No maximum-settings promise or assumed universal iOS memory budget.

How quality decisions will be made

Change	Main evidence to compare
Resolution / effects	GPU frame cost, slow frames, render-target memory and visible benefit.
Pedestrians / traffic	CPU time, engine heap, streaming demand, behavior and control response.
Textures / draw distance	Memory peaks, uploads, I/O stalls, repeated travel and visible benefit.

The successful browser session shows that improvement is worth investigating. It does not establish how much native headroom remains in a sustained open-world workload.

RULES FOR EVERY REMAINING STAGE

Execution, diagnostics and delivery

Finish faster by following the critical path

Focus on required Prologue startup and scene behavior, then controls/audio and the playable route. Defer networking, unrelated optional services, broad controller coverage, cosmetic UI and high settings. Reuse the existing engine, original shaders and content-verified compiler objects; avoid speculative rewrites.

During iteration, reproduce the actual failure with a representative command/asset fixture and test the affected subsystem. Before delivery, run the relevant regression gate and IPA audit. Phone tests close device/visual/playable gates; fixtures and Mac runs cannot replace them.

Retain useful, bounded diagnostics

Area	Record and use
GPU failure	Opcode/payload, shader IDs, resource/view/format details, negotiated capabilities, validation message and last completed operation.
Startup / loading	Current phase, worker and mailbox state, pending assets, delivered bytes, missing files, metadata classes, script progress and camera starts.
Performance / memory	Frame-time samples, CPU/GPU waits, process peaks/trends, engine heap, GPU/staging/cache lifetime and thermal state.
Verification	Visible scene and animation, control response, audio progression, tested route/duration, restart and pause/resume behavior.

Known uncertainty and the next decision

The immediate decision is the 0.18 physical scene result. Resolve any newly exposed required operation and retest that boundary. The 111 untranslated shader records and remaining guarded graphics behaviors require investigation if reached. Scenario/defaulttasks metadata, missing banks and offline warnings need diagnosis by effect on the scene or mission; successful reads do not prove every expected asset is present.

A native app still has iOS memory and hardware limits. There is no reliable completion date yet, and an exhaustive prediction of runtime failures is not possible. Re-estimate remaining work after the first verified scene, then after input/audio and the sustained route.

Isolation and release discipline

Work only in the native prototype and independent work directories. Leave both protected Desktop PC folders, the old offline package and deployed main unchanged. Publish native source only to native-ios-prototype. Preserve prior IPAs. GitHub Actions produces unsigned ARM64 IPAs for the established local signing/install route; account credentials stay outside the repository.

Evidence used for this roadmap

Project records, verified 14 October 2026: ROADMAP.txt; VERIFICATION-0.18.txt; AGENTS.md; the reported 0.17 iPhone trace. These records describe completed checks and pending gates; this PDF does not perform new engine or device testing.

Native 0.18 successful CI run | Source revision: 4096ea53c868bfc4d45b751362a6e336b58101f6.

Status: (stage 3 in progress) (5 stages left). Advance the stage count only when that stage's verification gate passes.